

Pixelated Image Abstraction

Timothy Gerstner^{1,*} Doug DeCarlo¹ Marc Alexa² Adam Finkelstein³ Yotam Gingold^{1,4} Andrew Nealen¹
¹Rutgers University ²TU Berlin ³Princeton University ⁴Columbia University

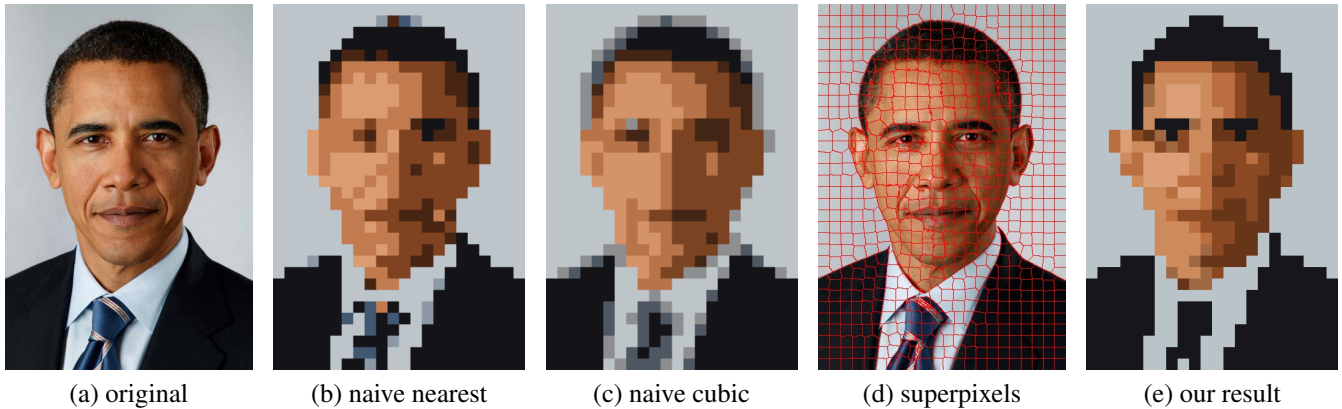


Figure 1: Pixel art images simultaneously use very few pixels and a tiny color palette. Attempts to represent image (a) using only 22×32 pixels and 8 colors using (b) nearest-neighbor or (c) cubic downsampling (both followed by median cut color quantization), result in detail loss and blurriness. We optimize over a set of superpixels (d) and an associated color palette to produce output (e) in the style of pixel art.

Abstract

We present an automatic method that can be used to abstract high resolution images into very low resolution outputs with reduced color palettes in the style of pixel art. Our method simultaneously solves for a mapping of features and a reduced palette needed to construct the output image. The results are an approximation to the results generated by pixel artists. We compare our method against the results of a naive process common to image manipulation programs, as well as the hand-crafted work of pixel artists. Through a formal user study and interviews with expert pixel artists we show that our results offer an improvement over the naive methods.

CR Categories: I.3.3 [Computer Graphics]: Picture/Image Generation—[I.3.4]: Computer Graphics—Graphics Utilities

Keywords: pixel art, image abstraction, non-photorealistic rendering, image segmentation, color quantization

Links:  DL  PDF

1 Introduction

We see pixel art every day. Modern day handheld devices such as the iPhone, Android devices and the Nintendo DS regularly utilize

pixel art to convey information on compact screens. Companies like Coca-Cola, Honda, Adobe, and Sony use pixel art in their advertisements [Vermehr et al. 2012]. It is used to make icons for desktops and avatars for social networks. While pixel art stems from the need to optimize imagery for low resolution displays, it has emerged as a contemporary art form in its own right. For example, it has been featured by MoMA, and there are a number of passionate online communities devoted to it. The “Digital Orca” by Douglas Coupland is a popular sight at the Vancouver Convention Center. France recently was struck by a “Post-it War”¹, where people use Post-It notes to create pixel art on their windows, competing with their neighbors across workplaces, small businesses, and homes.

What makes pixel art both compelling and difficult is the limitations imposed on the medium. With a significantly limited palette and resolution to work with, the task of creating pixel art becomes carefully choosing the set of colors and placing each pixel such that the final image best depicts the original subject. This task is particularly difficult as pixel art is typically viewed at a distance where the pixel grid is clearly visible, which has been shown to contribute to the perception of the image [Marr and Hildreth 1980]. As seen in Figure 2, creating pixel art is not a simple mapping process. Features such as the eyes and mouth need to be abstracted and resized in order to be represented in the final image. The end product, which is no longer physically accurate, still gives the impression of an identifiable person.

However, few, if any methods exist to automatically create effective pixel art. Existing downsampling methods, two of which are shown in Figure 1, do not accurately capture the original subject. Artists often turn to making pieces by hand, pixel-by-pixel, which can take a significant amount of time and requires a certain degree of skill not easily acquired by novices of the art. Automated and semi-automated methods have been proposed for other popular art forms, such as line drawing [DeCarlo et al. 2003; Judd et al. 2007] and painting [Gooch et al. 2002]. Methods such as [DeCarlo and Santella 2002] and [Winnemöller et al. 2006] not only abstract images, but do so while retaining salient features.

¹<http://www.postitwar.com/>

*e-mail:timgerst@cs.rutgers.edu

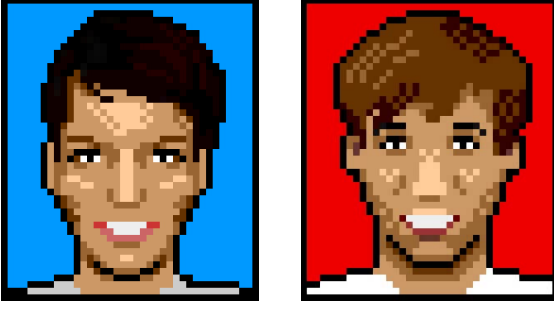


Figure 2: “Alice Blue” and “Kyle Red” by Alice Bartlett. Notice how faces are easily distinguishable even with this limited resolution and palette. The facial features are no longer proportionally accurate, similar to deformation in a caricature.

We introduce an automated process that transforms high resolution images into low resolution, small palette outputs in a pixel art style. At the core of our algorithm is a multi-step iterative process that simultaneously solves for a mapping of features and a reduced palette to convert an input image to a pixelated output image. In the first part of each iteration we use a modified version of an image segmentation proposed by Achanta et al. [2010] to map regions of the input image to output pixels. In the second step, we utilize an adaptation of mass-constrained deterministic annealing [Rose 1998] to find an optimal palette and its association to output pixels. These steps are interdependent, and the final solution is an optimization of both the physical and palette sizes specified by the user. Throughout this process we utilize the perceptually uniform CIELAB color space [Sharma and Trussell 1997]. The end result serves as an approximation to the process performed by pixel artists. Aside from assisting a class of artists in this medium, applications for this work include automatic and semi-automatic design of low-resolution imagery in handheld, desktop, and online contexts like Facebook and Flickr, wherever iconic representations of high-resolution imagery are used.

2 Related Work

One aspect of our problem is to reproduce an image as faithfully as possible while constrained to just a few output colors. Color quantization is a classic problem wherein a limited color palette is chosen based on an input image for indexed color displays. A variety of methods were developed in the 1980’s and early 1990’s prior to the advent of inexpensive 24-bit displays, for example [Gervautz and Purgathofer 1990; Heckbert 1982; Orchard and Bouman 1991; Wu 1992]. A similar problem is that of selecting a small set of custom inks to be used in printing an image [Stollnitz et al. 1998]. These methods rely only on the color histogram of the input image, and are typically coupled to an independent dithering (or halftoning) method for output in a relatively high resolution image. In our problem where the spatial resolution of the output is also highly constrained, we optimize simultaneously the selection and placement of colors in the final image.

The problem of image segmentation has been extensively studied. Proposed solutions include graph-cut techniques, such as the method proposed by Shi and Malik [1997], and superpixel-based methods QuickShift [Vedaldi and Soatto 2008], Turbopixels [Levinshtein et al. 2009], and SLIC [Achanta et al. 2010]. In particular, SLIC (Simple Linear Iterative Clustering) produces regular sized and spaced regions with low computational overhead given very few input parameters. These characteristics make SLIC an appropriate starting point for parts of our method.

Mass-constrained deterministic annealing (MCDA) [Rose 1998] is a method that uses a probabilistic assignment while clustering. Similar to k-means, it uses a fixed number of clusters, but unlike k-means it is independent of initialization. Also, unlike simulated annealing [Kirkpatrick et al. 1983], it does not randomly search the solution space and will converge to the same result every time. We use an adapted version of MCDA for color palette optimization.

Puzicha et al. [2000] proposed a method that reduces the palette of an image and applies half-toning using a model of human visual perception. While their method uses deterministic annealing and the CIELAB space to find a solution that optimizes both color reduction and dithering, our method instead emphasizes palette reduction in parallel with the reduction of the output resolution.

Kopf and Lischinski [2011] proposed a method that extracts vector art representations from pixel art. This problem is almost the inverse of the one presented in this paper. However, while their solution focuses on interpolating unknown information, converting an image to pixel art requires compressing known information.

Finally, we show that with minor modification our algorithm can produce “posterized” images, wherein large regions of constant color are separated by vectorized boundaries. To our knowledge, little research has addressed this problem, though it shares some aesthetic concerns with the *artistic thresholding* approach of Xu and Kaplan [2007].

3 Background

Our technique for making pixel art builds upon two existing techniques, which we briefly describe in this section.

SLIC. Achanta et al. [2010] proposed an iterative method to segment an image into regions termed “superpixels.” The algorithm is analogous to k-means clustering [MacQueen 1967] in a five dimensional space (three color and two positional), discussed for example in Forsyth and Ponce [2002]. Pixels in the input image p_i are assigned to superpixels p_s by minimizing

$$d(p_i, p_s) = d_c(p_i, p_s) + m\sqrt{\frac{N}{M}}d_p(p_i, p_s) \quad (1)$$

where d_c is the color difference, d_p is the positional difference, M is the number of pixels in the input image, N is the number of superpixels, and m is some value in the range $[0, 20]$ that controls the relative weight that color similarity and pixel adjacency have on the solution. The color and positional differences are measured using Euclidean distance (as are all distances in our paper, unless otherwise noted), and the colors are represented in LAB color space. Upon each iteration, superpixels are reassigned to the average color and position of the associated input pixels.

Mass Constrained Deterministic Annealing. MCDA [Rose 1998] is a global optimization method for clustering that draws upon an analogy with the process of annealing a physical material. We use this method both for determining the colors in our palette, and for assigning one of these palette colors to each pixel—each cluster corresponds to a palette color.

MCDA is a fuzzy clustering algorithm that probabilistically assigns objects to clusters based on their distance from each cluster. It relies on a temperature value T , which can be viewed as proportional to the expected variance of the clusters. Initially, T is set to a high value T_0 , which makes each object equally likely to belong to any cluster. Each time the system locally converges T is lowered (and the variance of each cluster decreases). As this happens, objects begin to prefer favor particular clusters, and as T approaches zero

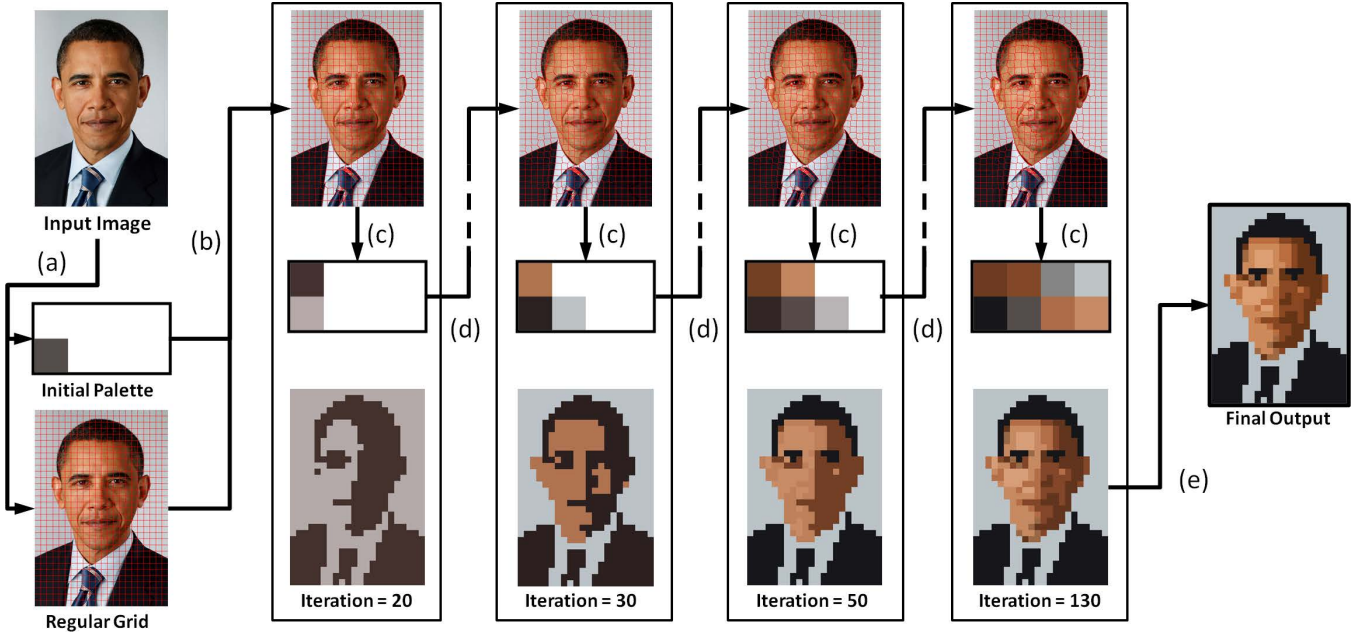


Figure 3: The pipeline of the algorithm. The superpixels (a) are initialized in a regular grid across the input image, and the palette is set to the average color of the M input pixels. The algorithm then begins iterating (b). Each iteration has two main steps: (c) the assignment of input pixels to superpixels, and (d) the assignment of superpixels to colors in the palette and updating the palette. This not only updates each color, but may also add new colors to the palette. After convergence, the palette is saturated (e) producing the final output.

each object becomes effectively assigned to a single cluster, at which point the final set of clusters is produced. In Section 4.3 we provide a formal definition of the conditional probability we use to assign superpixels to colors in the palette.

Since at high T having multiple clusters is redundant, MCDA begins with a single cluster, represented internally by two sub-clusters. At the beginning of each iteration these sub-clusters are set to slight permutations of their mean. At a high T these clusters converge to the same value after several iterations, but as the temperature is lowered they begin to naturally separate. When this occurs, the cluster is split into two separate clusters (each represented by their own sub-clusters). This continues recursively until the (user specified) maximum number of clusters is reached.

4 Method

Our algorithm is an iterative procedure—an example execution is shown in Figure 3. The process begins with an input image of width w_{in} and height h_{in} and produces an output image of width w_{out} and height h_{out} which contains at most K different colors—the palette size. Given the target output dimensions and palette size, each iteration of the algorithm segments the pixels in the input into regions corresponding to pixels in the output and solves for an optimal palette. Upon convergence, the palette is saturated to produce the final output. In this section, we describe our algorithm in terms of the following:

Input Pixels The set of pixels in the input image, denoted as p_i where $i \in [1, M]$, and $M = w_{in} \times h_{in}$.

Output Pixels The set of pixels in the output image, denoted as p_o where $o \in [1, N]$, and $N = w_{out} \times h_{out}$.

Superpixel A region of the input image, denoted as p_s where $s \in [1, N]$. The superpixels are a partition of the input image.

Palette A set of K colors c_k , $k \in [1, K]$ in LAB space.

Our algorithm constructs a mapping for each superpixel that relates a region of input pixels with a single pixel in the output, as in Figure 4. The algorithm proceeds similarly to MCDA, with a superpixel refinement and palette association step performed upon each iteration, as summarized in Algorithm 1. Section 4.5 describes how the algorithm can be expanded to allow a user to indicate important regions in the input image.

Algorithm 1

- ▷ **initialize** superpixels, palette and temperature T (Section 4.1)
- ▷ **while** ($T > T_f$)
 - ▷ **refine** superpixels with 1 step of modified SLIC (Section 4.2)
 - ▷ **associate** superpixels to colors in the palette (Section 4.3)
 - ▷ **refine** colors in the palette (Section 4.3)
 - ▷ **if** (palette converged)
 - ▷ **reduce** temperature $T = \alpha T$
 - ▷ **expand** palette (Section 4.3)
- ▷ post-process (Section 4.4)

4.1 Initialization

The N superpixel centers are initialized in a regular grid across the input image, and each input pixel is assigned to the nearest superpixel (in (x, y) space, measured to the superpixel center). The palette is initialized to a single color, which is set to the mean value of the M input pixels. All superpixels are assigned this mean color. See Figure 3, step (a).

The temperature T is set to $1.1T_c$, where T_c is the critical temperature of the set of M input pixels, defined as twice the variance along the major principal component axis of the set in LAB space [Rose 1998]. The T_c of a set of objects assigned to a cluster is the temperature at which a cluster will naturally split. Therefore, this policy ensures that the initial temperature is easily above the temperature at which more than one color in the palette would exist.

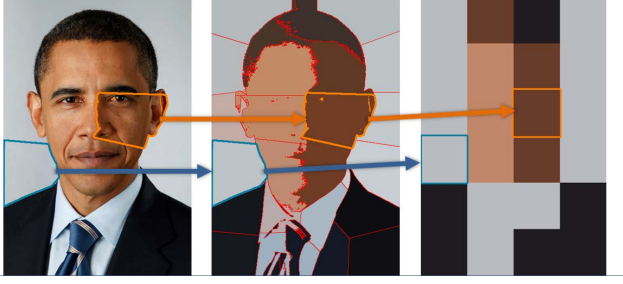


Figure 4: Pixels in the input image (left) are associated with superpixel regions (middle). Each superpixel region corresponds to a single pixel in the output image (right).

4.2 Superpixel refinement

This stage of the algorithm assigns pixels in the input image to superpixels, which correspond to pixels in the output image—see steps (b) and (d) in Figure 3.

To accomplish this task, we use a single iteration of our modified version of SLIC. In the original SLIC algorithm, upon each iteration, every input pixel is assigned to the superpixel that minimizes $d(p_i, p_s)$, and the color of each superpixel is set to the mean color value of its associated input pixels, m_s . However, in our implementation, the color of each superpixel is set to the palette color that is associated with the superpixel (the construction of this mapping is explained in Section 4.3). This interdependency with the palette forces the superpixels to be optimized with respect to the colors in the palette rather than the colors in the input image. Figure 5 shows the results of using the mean color value instead of our optimized palette used in Figure 1.

However, this also means the color error will be generally higher. As a result, we’ve found that minimizing $d(p_i, p_s)$ using a value of $m = 45$ is more appropriate in this case (Achanta et al. [2010] suggest $m = 10$). This increases the weight of the positional distance and results in a segmentation that contains superpixels with relatively uniform size.

Next, we perform two steps, one modifies each superpixel’s (x, y) position for the next iteration, and one changes each superpixel’s representative color. Each step is an additional modification to the original SLIC method and significantly improves the final result.

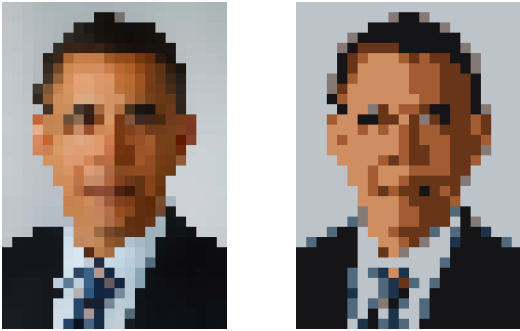


Figure 5: Our method uses palette colors when finding superpixels. Using the mean color of a superpixel works when the palette is unconstrained (left), but fails when using a constrained palette (right). This is because the input pixels cluster into superpixels based on colors that do not exist in the final image, which creates a discrepancy. Using the palette colors to represent the superpixels removes this discrepancy.

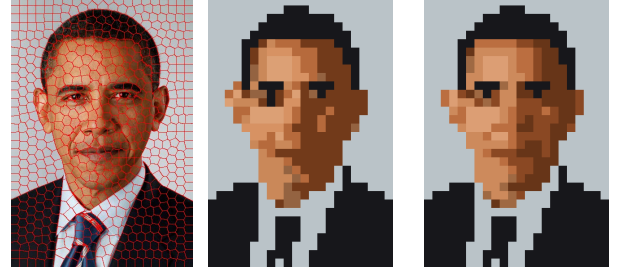


Figure 6: Without the Laplacian smoothing step, the superpixels (left) tend to have 6-connected neighborhoods. This causes small distortions in the output (center), which are particularly noticeable on the ear, eye and mouth, when compared to original output that uses the superpixels that included the smoothing step (right).

As seen in Figure 6 (left), SLIC results in superpixel regions which tend to be organized in 6-connected neighborhoods (i.e. a hexagonal grid). This is caused by how the (x, y) position of each superpixel is defined as the average position of the input pixels associated with it. This hexagonal grid does not match the neighborhoods of the output pixels, which are 8-connected (i.e. a rectangular grid) and will give rise to undesirable distortions of image features and structures in the output, as seen in Figure 6(center).

We address this problem with Laplacian smoothing. Each superpixel center is moved a percentage of the distance from its current position to the average position of its 4-connected neighbors (using the neighborhoods at the time of initialization). We use 40%. As seen in Figure 1 (d), this improves the correspondence between the superpixel and output pixel neighborhoods. Specifically, it helps ensure that superpixel regions that are adjacent in the input map are also adjacent pixels in the output. To be clear, it is only in the next iteration when the superpixels will be reassigned based on this new center, due to the interleaved nature of our algorithm.

In our second additional step, the color representatives of the superpixels are smoothed. In the original SLIC algorithm, the representative color for each superpixel is the average color m_s of the input pixels associated with it. However, simply using the mean color can become problematic for continuous regions in the image that contain a color gradient (such as a smooth shadowed surface). While this gradient appears natural in the input image, the region will not appear continuous in the pixelated output.

To remedy this, our algorithm adjusts the values of m_s using a bilateral filter. We construct an image of size $w_{out} \times h_{out}$ where each superpixel is assigned the same position as its corresponding output pixel, with value m_s . The colors that results from bilaterally filtering this image, m_s' are used while iterating the palette.

4.3 Palette refinement

Palette iteration is performed using MCDA [Rose 1998]. Each iteration of the palette, as seen in step (c) in Figure 3, can be broken down into three basic steps: **associating** superpixels to colors in the palette, **refining** the palette, and **expanding** the palette. The associate and refine steps occur every iteration of our algorithm. When the palette has converged for the current temperature T , the expand step is performed.

It is important to note how we handle the sub-clusters mentioned in Section 3: we treat each sub-cluster as a separate color in the palette, and keep track of the pairs. The color of each c_k is the average color of its two sub-clusters. When the maximum size of the palette is reached (in terms of the number of distinct colors c_k),

we eliminate the sub-clusters and represent each color in the palette as a single cluster.

Associate. The MCDA algorithm requires a probability model that states how likely a particular superpixel will be associated with each color in the palette. See Figure 7. The conditional probability $P(c_k|p_s)$ of a superpixel p_s being assigned color c_k depends on the color distance in LAB space and the current temperature, and is given by (after suitable normalization):

$$P(c_k|p_s) \propto P(c_k) e^{-\frac{\|m_s' - c_k\|}{T}} \quad (2)$$

$P(c_k)$ is the probability that color c_k is assigned to any superpixel, given the existing assignment. Upon initialization, there is only one color, and thus this value is initialized to 1. As more colors are introduced into the palette, the value of this probability is computed by marginalizing over p_s :

$$P(c_k) = \sum_{s=1}^N P(c_k|p_s)P(p_s) \quad (3)$$

For the moment, $P(p_s)$ simply has a uniform distribution. This will be revisited in Section 4.5 when incorporating user-specified importance. The values of $P(c_k)$ are updated after the values of $P(c_k|p_s)$ are computed using Equation 2. Each superpixel is assigned to the color in the palette that maximizes $P(c_k|p_s)$. Intermediate results of this assignment can be seen in Figure 3 (bottom row). The exponential distribution in Equation 2 tends towards a uniform distribution for large values of T , in which case each superpixel will be evenly associated with every palette color. As T decreases, superpixels favor colors in the palette that are less distant. At the final temperature, the generic situation after convergence has $P(c_k|p_s) = 1$ for a single color in the palette and $P(c_k|p_s) = 0$ for the rest. In this case, deterministic annealing is equivalent to k-means clustering.

Refine. The next step is to refine the palette by reassigning each color c_k to a weighted average of all superpixel colors, using the probability of association with that color:

$$c_k = \frac{\sum_{s=1}^N m_s' P(c_k|p_s)P(p_s)}{P(c_k)} \quad (4)$$

This adapts the colors in the existing palette given the revised superpixels. Such changes in the palette can be seen in Figure 3, as the computation progresses.

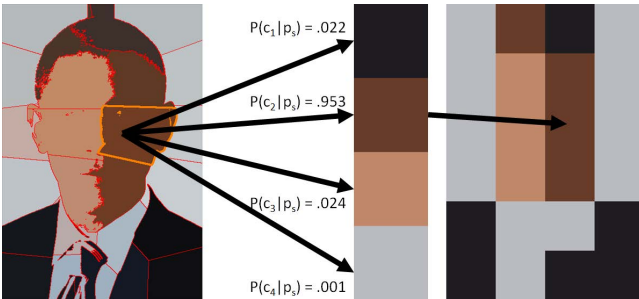


Figure 7: Each superpixel (left) is associated by some conditional probability $P(c_k|p_s)$ to each color in the palette (middle). The color with the highest probability is assigned to the superpixel and its associated output pixel in the final image (right).

Expand. Expansion only occurs during an iteration if the palette has converged for the current temperature T (convergence is measured by the total change in the palette since last iteration being less than some small value $\epsilon_{\text{palette}}$). First, the temperature is lowered by some factor α (we use 0.7). Next, the palette is expanded if the number of colors is less than the number specified by the user. For each c_k we check to see if the color needs to be split into two separate colors in the palette. As per MCDA, each color in the palette is represented by two cluster points c_{k_1} and c_{k_2} . We use $\|c_{k_1} - c_{k_2}\| > \epsilon_{\text{cluster}}$ (where $\epsilon_{\text{cluster}}$ is a sufficiently small number), to check for palette separation. If so, the two cluster points are added to the palette as separate colors, each with its own pair of cluster points. As seen in Figure 3, over the course of many iterations, the palette grows from a single color to a set of eight (which is the maximum number specified by the user in this example).

After resolving any splits, each color is represented by two sub-clusters with the same value (unless the maximum number of colors have been reached). In order for any color's sub-clusters to separate in the following iterations, c_{k_1} and c_{k_2} must be made distinctly different. To do so, we perturb the sub-clusters of each color by a small amount along the principal component axis of the cluster in LAB space. Rose [1998] has shown this to be the direction a cluster will split. This perturbation allows the sub-clusters of each color to merge when $T > T_c$ and separate when $T < T_c$.

Algorithm 1 is defined so that the superpixel and palette refinement steps are iterated until convergence. The system converges when the temperature has reached the final temperature T_f and the palette converges. We use $T_f = 1$ to avoid truncation errors as the exponential component of the Equation 2 becomes small.

4.4 Palette Saturation

As a post-processing step, we provide the option to saturate the palette, which is a typical pixel artist technique, by simply multiplying the a and b channels of each color by a parameter $\beta > 1$. This value used in all our results is $\beta = 1.1$. Lastly, by converting to from LAB to RGB space, our algorithm outputs the final image.

4.5 Importance Map

As stated, our method does not favor any image content. For instance, nothing is in place that can distinguish between foreground and background objects, or treat them separately in the output. However, user input (or the output of a computer vision system) can easily be incorporated into our algorithm to prioritize the foreground in the output. Thus, our system allows additional input at the beginning of our method. Users can supply a $w_{\text{in}} \times h_{\text{in}}$ grayscale image of weights $W_i \in [0, 1]$, $i \in [1, M]$, used to indicate the importance of each input pixel p_i . We incorporate this map when iterating the palette (Section 4.3) by adjusting the prior $P(p_s)$. Given the importance map, the value $P(p_s)$ for each superpixel is given by the average importance of all input pixels contained in superpixel p_s (and suitable normalization across all superpixels):

$$P(p_s) \propto \frac{1}{|p_s|} \sum_{\text{pixel } i \in p_s} W_i \quad (5)$$

$P(p_s)$ thus determines how much each superpixel affects the resulting palette, through Equations 3 and 4. This results in a palette that can better represent colors in the regions of the input image marked as important.

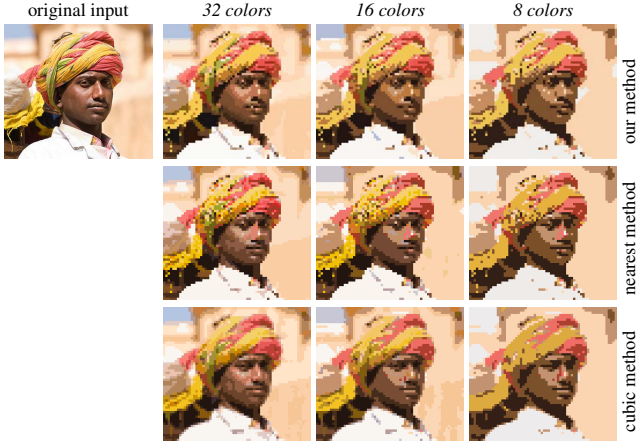


Figure 8: Varying the palette size (output images are 64×58).

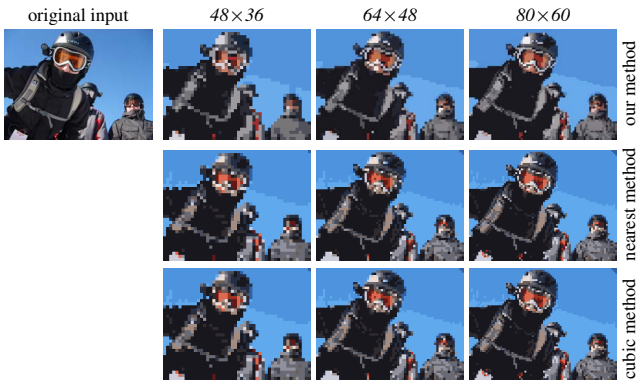


Figure 9: Varying the output resolution (palette has 16 colors).

5 Results

We tested our algorithm on a variety of input images at various output resolutions and color palette sizes (Figures 8–13). For each example, we compare *our method* to two naive approaches:

- *nearest method*: a bilateral filter followed by median cut color quantization, followed by nearest neighbor downsampling,
- *cubic method*: cubic downsampling followed by median cut color quantization.

All of our results use the parameter settings from Section 4. Each result was produced in generally less than a minute on an Intel 2.67Ghz i7 processor with 4GB memory. Each naive result is saturated using the same method described in Section 4.4. Please note it is best to view the results up-close or zoomed-in, with each pixel being distinctly visible.

In Figure 8, we show the effects of varying the number of colors in the output palette. Our method introduces fewer isolated colors than the nearest method, while looking less washed out than the cubic method. As the palette size shrinks, our method is better able to preserve salient colors, such as the green in the turban. Our method’s palette assignment also improves the visibility of the eyes and does not color any of the face pink.

Similar results are seen in Figure 9 when we vary the output resolution. Again we see that the cubic method produces washed-out images and the nearest method has a speckled appearance. At

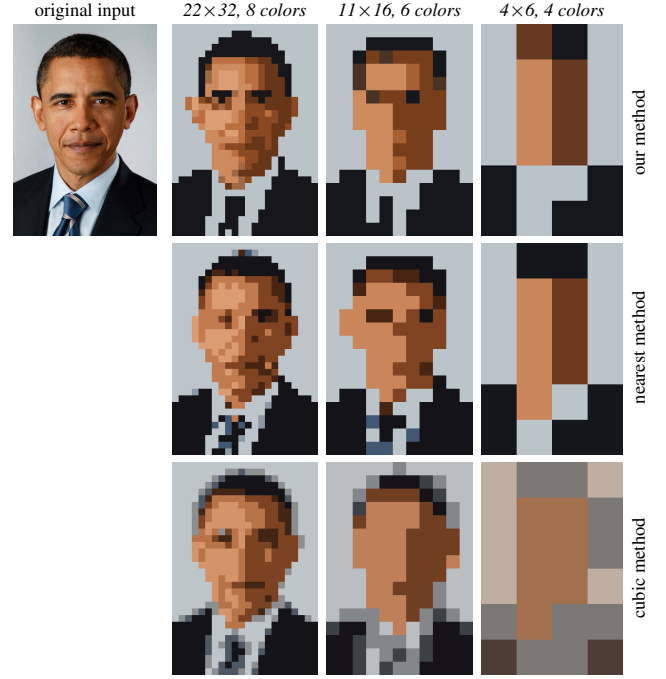


Figure 10: Examples of very low resolution and small palette sizes.

all resolutions, our method preserves features such as the goggles more faithfully, and consistently chooses more accurate skin tones for the faces, whereas both naive methods choose gray.

Using our technique, the image of Barack Obama is recognizable even at extremely small output resolutions and palette sizes (Figure 10). At 22×32 and 11×16 , our method more clearly depicts features such as the eyes while coloring regions such as the hair and tie more consistently. At 11×16 , the nearest method produces a result that appears to distort facial features, while the cubic method produces a result that “loses” the eyes. At 6×4 , results are very abstract, but our method’s output could still be identified as a person or as having originated from the input.

In Figure 11, we compare our output to manual results created by expert pixel artists. While our results exhibit the same advantages seen in the previous figures over the naive methods, they do not match the results made by artists. Expert artists are able to heavily leverage their human understanding of the scene to emphasize and de-emphasize features and make use of techniques such as dithering and edge highlighting. While there are many existing methods to automatically dither an image, at these resolutions the decision on when to apply dithering is nontrivial, and uniform dithering can introduce undesired textures to surfaces (such as skin).

In Figure 12, we present results from our method using an input importance map. For both examples, the importance map emphasizes the face and de-emphasizes the background; consequently, more colors are allocated to the face in each example at the expense of the background.

Figure 13 contains additional results computed using various input images. Overall, our approach is able to produce less noisy, sharper images with a better selection of colors than the naive automatic techniques we compared against.

To verify our analysis, we conducted a formal user study with 100 subjects using Amazon Mechanical Turk. Subjects were shown the original image and the results of our method and the two naive

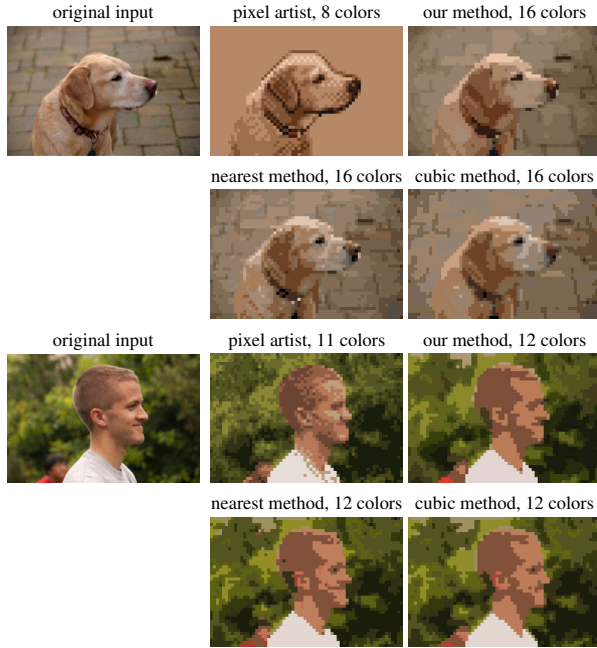


Figure 11: Comparing to the work of expert pixel artists (64×43)

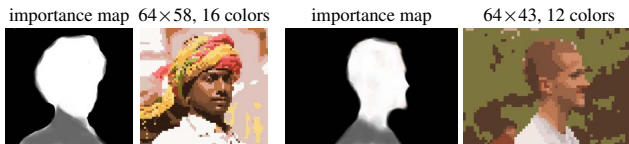


Figure 12: Results using an importance map.

methods. The results were scaled to approximately 256 pixels along their longest dimension using nearest neighbor upsampling, so that user’s could clearly see the pixel grid. We asked subjects the question, “Which of the following best represents the image above?” Subjects responded by choosing a result image. The stimulus sets and answer choices were randomized to remove bias. The study consisted of the example images and parameters shown in our paper, excluding the results generated using user-markup, and each stimulus was duplicated four times (sixty total).

We accounted for users answering randomly by eliminating the results of any subject who gave inconsistent responses (choosing the same answer for less than three of the four duplicates) on more than a third of the stimuli. This reduced the number of valid responses to forty. The final results show that users choose our results 41.49% of the time, the nearest method 34.52% of the time, and the cubic method 23.99% of the time. Using a one-way analysis of variance (ANOVA) on the results, we found a p value of 2.12×10^{-6} , which leads us to reject the null hypothesis that subjects all chose randomly. Using Tukey’s range test we found that our method is significantly different from the nearest method with a 91% confidence interval, and from the cubic method with a 99% confidence interval. While we acknowledge that the question asked is difficult one given that it is an aesthetic judgment, we believe the results of this study still show subjects prefer the results of our method over the results of either naive method.

We also received feedback from three expert pixel artists on our results; each concluded that our results are, in general, an improvement over the naive approaches. Ted Martens, creator of the Pixel Fireplace, said that our algorithm “chooses better colors for the

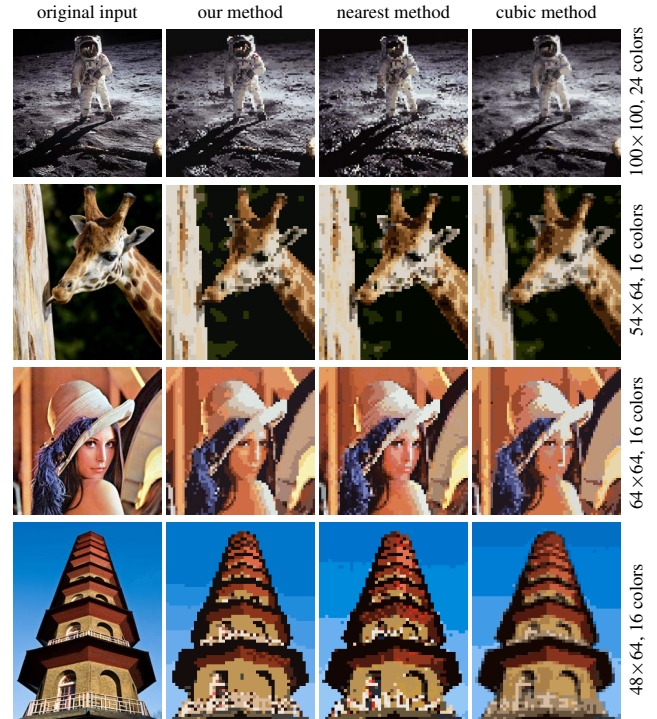


Figure 13: Additional results.

palette, groups them well, and finds shapes better.” Adam Saltsman, creator of Canabalt and Flixel, characterized our results as “more uniform, more reasonable palette, better forms, more readable.” Craig Adams, art director of Superbrothers: Sword & Sworcery EP, observed that “essential features seem to survive a bit better [and] shapes seem to come through a bit more coherently. I think the snowboarder’s goggles are the clearest example of an essential shape—the white rim of the goggle—being coherently preserved in your process, while it decays in the ‘naive’ process.”

Finally, while not the direct goal of our work, we briefly mention a secondary application of our method, image *posterization*. This artistic technique uses just a few colors (originally motivated by the use of custom inks in printing) and typically seeks a vectorized output. LiveTrace, a feature in Adobe Illustrator can posterize the image in Figure 1(a), yielding Figure 14(a) with only 6 colors. A simple modification to our optimization that omits the smoothing step (Figure 6-left) and then colors the original image via associated superpixels gives us Figure 14(b), which makes a more effective starting point for vectorization. The resulting Figure 14(c) offers improved spatial and color fidelity, based on a good faith effort to produce a similar style in Illustrator.

6 Conclusion, Limitations and Future work

We present a multi-step iterative process that simultaneously solves for a mapping of features and a reduced palette to convert an input image to a pixelated output image. Our method shows several advantages over naive methods. Our results have a more vibrant palette, retain more features of the original image, and produce a cleaner output with fewer artifacts. While the naive methods produce unidentifiable results at very low resolutions and palette sizes, our approach is still able to create iconic images that conjure the original image. Thus our method makes a significant step towards the quality of images produced by pixel artists.

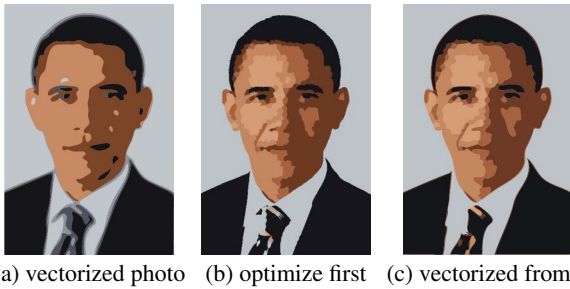


Figure 14: (a) Photo posterized with Illustrator (6 colors). (b) Optimization without Laplacian smoothing, coloring associated input pixels (6 colors). (c) Vectorizing b in Illustrator yields similar style with better spatial and color fidelity than a.

Nevertheless, our method has several limitations which we view as potential avenues for future research. While pixel artists view our results as an improvement, they also express the desire to have a greater control over the final product. We partially address this by incorporating an importance map that guides palette selection, but there is still room to incorporate additional choices typically made by a pixel artist. We wish to expand our proposed method to include the user “in the loop” while generating the final output. We intend to investigate an interface that will allow users to guide the process and introduce additional techniques used by pixel artists, such as edge highlighting. Likewise the ability for the user to specify colors in the palette would broaden the potential applications of this work to include, for example, reproduction of an image in repeating tiles like Lego, or design for architectural facades composed of particular building materials like known shades of brick.

Another limitation that we would like to address in future work is the use of selective dithering. Our current framework avoids the need for dithering by automatically selecting palette colors that express the most critical shades in the source image. However, artists often use dithering to simultaneously achieve blended colors and express texture (for example, the dog in Figure 11). This kind of effect could be incorporated either by allowing user to specify dither regions or through texture analysis, but in either case would make the optimization process more complex.

7 Acknowledgments

We thank the anonymous reviewers for their helpful feedback. We also wish to thank pixel artists Craig Adams, Ted Martens, and Adam Saltsman for their advice and comments. This research is supported in part by the NSF (IIS-09-16845 and DGE-05-49115). The following copyrighted images are used with permission: Figure 2 by Alice Bartlett, Figure 8 by Louis Vest, Figure 13 (giraffe) by Paul Adams, and Figure 13 (pagoda) by William Warby. The pixel art in Figure 11 is copyright Adam Saltsman (top) and Ted Martens (bottom).

References

- ACHANTA, R., SHAJI, A., SMITH, K., LUCCHI, A., FUA, P., AND SÜSSTRUNK, S. 2010. SLIC Superpixels. Tech. rep., IVRG CVLAB.
- DECARLO, D., AND SANTELLA, A. 2002. Stylization and abstraction of photographs. *ACM Trans. Graph.* 21, 769–776.
- DECARLO, D., FINKELSTEIN, A., RUSINKIEWICZ, S., AND SANTELLA, A. 2003. Suggestive contours for conveying shape. *ACM Trans. Graph.* 22, 3 (July), 848–855.
- FORSYTH, D. A., AND PONCE, J. 2002. *Computer Vision: A Modern Approach*. Prentice Hall.
- GERVAUTZ, M., AND PURGATHOFER, W. 1990. Graphics gems. ch. A simple method for color quantization: octree quantization, 287–293.
- GOOCH, B., COOMBE, G., AND SHIRLEY, P. 2002. Artistic vision: painterly rendering using computer vision techniques. In *Non-Photorealistic Animation and Rendering (NPAR)*, 83–90.
- HECKBERT, P. 1982. Color image quantization for frame buffer display. *SIGGRAPH Comput. Graph.* 16 (July), 297–307.
- JUDD, T., DURAND, F., AND ADELSON, E. H. 2007. Apparent ridges for line drawing. *ACM Trans. Graph.* 26, 3, 19.
- KIRKPATRICK, S., GELATT, C. D., AND VECCHI, M. P. 1983. Optimization by simulated annealing. *Science* 220, 671–680.
- KOPF, J., AND LISCHINSKI, D. 2011. Depixelizing pixel art. *ACM Trans. Graph.* 30, 4, 99:1–99:8.
- LEVINSHTAIN, A., STERE, A., KUTULAKOS, K. N., FLEET, D. J., DICKINSON, S. J., AND SIDDIQI, K., 2009. Turbopixels: Fast superpixels using geometric flows.
- MACQUEEN, J. B. 1967. Some methods for classification and analysis of multivariate observations. In *Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability*, 281–297.
- MARR, D., AND HILDRETH, E. 1980. Theory of edge detection. *International Journal of Computer Vision*.
- ORCHARD, M., AND BOUMAN, C. 1991. Color quantization of images. *IEEE Trans. on Signal Processing* 39, 2677–2690.
- PUZICHA, J., HELD, M., KETTERER, J., BUHMANN, J. M., MEMBER, IEEE, AND FELLNER, D. W. 2000. On spatial quantization of color images. *IEEE Transactions on Image Processing* 9, 666–682.
- ROSE, K. 1998. Deterministic annealing for clustering, compression, classification, regression, and related optimization problems. *Proceedings of the IEEE* 86, 11 (nov), 2210–2239.
- SHARMA, G., AND TRUSSELL, H. J. 1997. Digital color imaging. *IEEE Transactions on Image Processing* 6, 901–932.
- SHI, J., AND MALIK, J. 1997. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22, 888–905.
- STOLLNITZ, E. J., OSTROMOUKHOV, V., AND SALESIN, D. H. 1998. Reproducing color images using custom inks. In *Proceedings of SIGGRAPH*, 267–274.
- VEDALDI, A., AND SOATTO, S. 2008. Quick shift and kernel methods for mode seeking. In *In European Conference on Computer Vision, volume IV*, 705–718.
- VERMEHR, K., SAUERTEIG, S., AND SMITAL, S., 2012. ebay. <http://hello.ebay.com>.
- WINNEMÖLLER, H., OLSEN, S. C., AND GOOCH, B. 2006. Real-time video abstraction. *ACM Trans. Graph.* 25, 1221–1226.
- WU, X. 1992. Color quantization by dynamic programming and principal analysis. *ACM Trans. Graph.* 11, 348–372.
- XU, J., KAPLAN, C. S., AND MI, X. 2007. Computer-generated papercutting. In *Computer Graphics and Applications, 2007. PG '07. 15th Pacific Conference on*, 343–350.